SISTEMAS DE TIEMPO REAL

Clase Planificación

Planificación de procesos - Tareas



Planificación de procesos - Tareas

- Tres tipos de tareas, según sus características temporales:

- Tareas periódicas. Ejecutan periódicamente en un periodo constante. Supondremos que todas las tareas con restricciones temporales son periódicas.
- Tareas esporádicas. Tienen plazo de finalización estricto, pero se ejecutan esporádicamente. Se tratan como tareas periódicas suponiendo que existe un tmín entre dos activaciones consecutivas. Este tiempo se tomará como el periodo "equivalente" de la tarea.
- Tareas aperiódicas. No tienen plazo de finalización, o si lo tienen, no es de obligado cumplimiento.

Otra clasificación de las tareas de TR en base a características semánticas:

- Tareas críticas. El fallo de una de estas tareas (por no ejecutarse a tiempo) puede ser catastrófico para el sistema.
- Tareas opcionales. Si se retrasa su ejecución no afecta a la seguridad del sistema. Realizan:
- Monitorización del sistema
- Tareas de mantenimiento
- Refina el resultado obtenido por tareas críticas

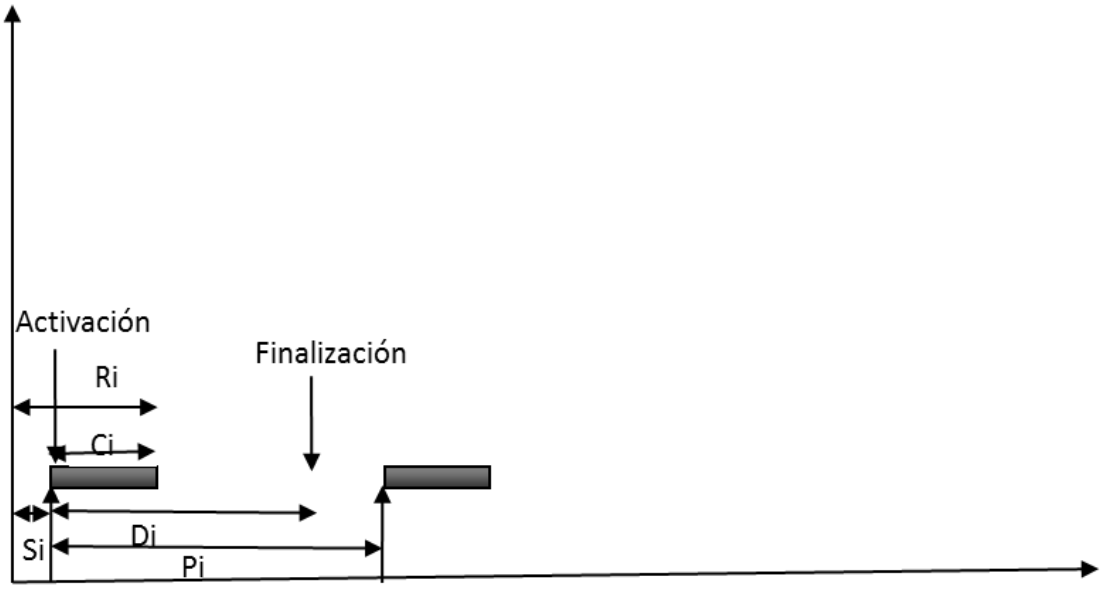
Las tareas opcionales se pueden dividir en dos grupos:

- Opcionales con plazo (hard aperiodic). Disponen de un tiempo de ejecución recomendado.
- Opcionales sin plazo (soft aperiodic).

Algunas políticas de planificación tienen en cuenta esta distinción e intentan cumplir los plazos de las tareas opcionales con plazo, siempre que no haya sobrecarga de tareas críticas o por el resto de tareas opcionales con plazo.

Parámetros

N: Número de tareas en el sistema
Pi: Periodo de activación
Ci: Tiempo máximo de ejecución
Di: Plazo máximo de terminación.
Ri: Tiempo de respuesta máximo. Es el tiempo desde que la tarea se activa hasta que termina realmente de ejecutarse.
Pri: Prioridad de la tarea (convenio: mayor valor, menor prioridad)
Si: Desfase respecto al momento inicial



Tiempo real crítico y opcional

- Tardanza de una tarea: diferencia entre el tiempo de respuesta y el plazo de ejecución de una tarea.

$$t_i = R_i - D_i$$

- Una tarea se ejecuta de forma tardía si su tardanza se hace positiva. O sea finaliza su ejecución después de haber vencido su plazo de ejecución.
- En los STR opcional un resultado puede ser útil, aunque sea tardío. Pero su utilidad decae.
- En tareas de tiempo real crítico la utilidad se hace 0 cuando la tardanza se hace positiva.
- La utilidad se expresaría en forma de un valor de la probabilidad de que el tiempo de respuesta sea menor o igual a la cota temporal.
- Un STR crítico sería aquel en que la probabilidad de que se cumplan las cotas temporales sea 1.

Tiempo real crítico y opcional

Teniendo en cuenta todos estos criterios, en los sistemas de tiempo real las restricciones temporales se pueden expresar en varios términos, por ejemplo:

1. **Restricciones deterministas.** Por ejemplo, el resultado que proporciona una tarea se debe obtener siempre antes de 50 ms.
2. **Restricciones probabilísticas.** Por ejemplo, la probabilidad de que el tiempo de respuesta exceda de 50 ms. debe ser inferior a 0.2.
3. **Restricciones en términos de una función de utilidad.** Por ejemplo, la utilidad de los resultados que proporciona una tarea debe ser 0.8 o superior.

Las restricciones temporales críticas raramente se expresan en los dos últimos términos, mientras que las restricciones temporales opcionales si suelen expresarse en términos de probabilidad o de utilidad.

Ejemplo: sistema telefónico, tras marcar el nro., la conexión se debe establecer en un tiempo que no sea excesivo, antes de 10 seg. en el 95% de los casos y antes de 20 seg. en el 99.95% de las llamadas

Tiempo de ejecución

- Tiempo de ejecución de una tarea variará entre un valor máximo y un mínimo, dependiendo de
  - ✓ saltos condicionales que se activen
  - ✓ iteraciones en bucles.
- Otros factores:
- Uso de recursos compartidos (comunicación, memoria virtual)
- Evitar uso de estructuras dinámicas que introducen variación en cantidad de memoria utilizada y tiempo para operar con estas estructuras.
- Es común que sólo una parte del sistema requiera comportamiento temporal estricto, y el resto opcional.
- Se puede considerar las tareas de tiempo real crítico, dejando el tiempo sobrante para el resto de tareas
- Se supone que el número de tareas de tiempo real crítico en el sistema es constante y conocido.

Tareas periódicas

- Para aumentar determinismo, se supone todas las tareas periódicas con un periodo de activación rigurosamente exacto, o el menor de los posibles
- También se pueden considerar como periódicas las tareas esporádicas, siempre que se pueda determinar un valor mínimo del tiempo entre dos activaciones consecutivas. Este valor mínimo será el que se toma como su periodo.

Factor de utilización

- Es la fracción de la CPU que se utiliza en el sistema

$$U = \sum_{i=1}^{N} \frac{Ci}{Pi}$$

- O sea la suma de las fracciones de CPU utilizado por el total de las tareas.
- Es necesario que el factor de utilización sea menor o igual que uno.

Hiperperiodo

- Es el mínimo común múltiplo de los periodos de todas las tareas
- A partir de este valor el sistema se repite, si se determina que un sistema es planificable en la duración de un hiperperiodo, lo será también durante toda la vida del sistema
- Con un número elevado de tareas el valor del hiperperiodo se dispara (en el caso más desfavorable es el producto de todos los periodos). Hay que tener en cuenta que la probabilidad de que dos números elegidos al azar sean primos es del 60%

Dependencia entre tareas

- Precedencia: una tarea no puede continuar hasta que otra la active explícitamente.
- Dependencia en datos: varias tareas utilizan datos que no pueden ser compartidos (exclusión mutua)
- Dependencia temporal relativa: Varias tareas deben finalizar sus trabajos en un instante que no puede distar más de una cantidad. El plazo no es absoluto sino relativo a otra tarea (videoconferencia en la que se quiere que la voz acompañe al movimiento de los labios)
- Restricciones de precedencia AND / OR: Es una relación entre tareas similar a la restricción de precedencia, pero cuando una tarea debe esperar a que finalicen varias predecesoras (AND), o sólo alguna de ellas (OR)
- Ejecución condicional: una tarea se ejecuta o no dependiendo del resultado de otra
- Relación de cauce: Es la relación típica del tipo productor / consumidor. El consumidor debe esperar a tener datos para continuar y el productor debe detenerse si el medio de comunicación se colapsa debido a que el productor envía información más rápido de lo que la lee el consumidor
- En el caso de que entre dos tareas exista una relación de precedencia, se supone ambas tareas como periódicas independientes con el mismo periodo igual al de la tarea que precede a la segunda

Parámetros funcionales

- Trabajos interrumpibles: pueden ser interrumpidos para ejecutar tareas más urgentes. La interrupción de un trabajo se llama preempción (preemption en inglés).
- No interrumpibles: ejecutar sin interrupción desde que comienza hasta que termina.
- Cuando se interrumpe tarea, el sistema debe guardar su estado (cambio de contexto). En el análisis se considera nulo el tiempo de cambio de contexto
- Importancia de los trabajos: El planificador para decidir el orden en que se ejecutan las tareas deberá tener en cuenta su importancia a través de una prioridad a las tareas
- Ejecución opcional: Si aparece sobrecarga se configuran tareas como opcionales, que podrán descartarse
- Si un trabajo de una tarea opcional se ejecute fuera de plazo, o no se ejecuta, producirá un funcionamiento degradado del sistema, pero seguirá ofreciendo un comportamiento satisfactorio
- Los trabajos obligatorios deben ejecutarse completamente dentro de sus plazos
- Se tendrá que utilizar un parámetro funcional que indique si un trabajo es obligatorio u opcional

Planificación

- Trabajos deben ser asignados a procesadores en un orden determinado
- Esto lo hace el planificador: decide que trabajos ejecuta cada procesador y en que orden
- Planificación es una asignación de recursos y tiempo a actividades de forma que se cumplan determinados requisitos de eficiencia.

– Estáticos sin tiempo real: minimizar tiempo de ejecución.

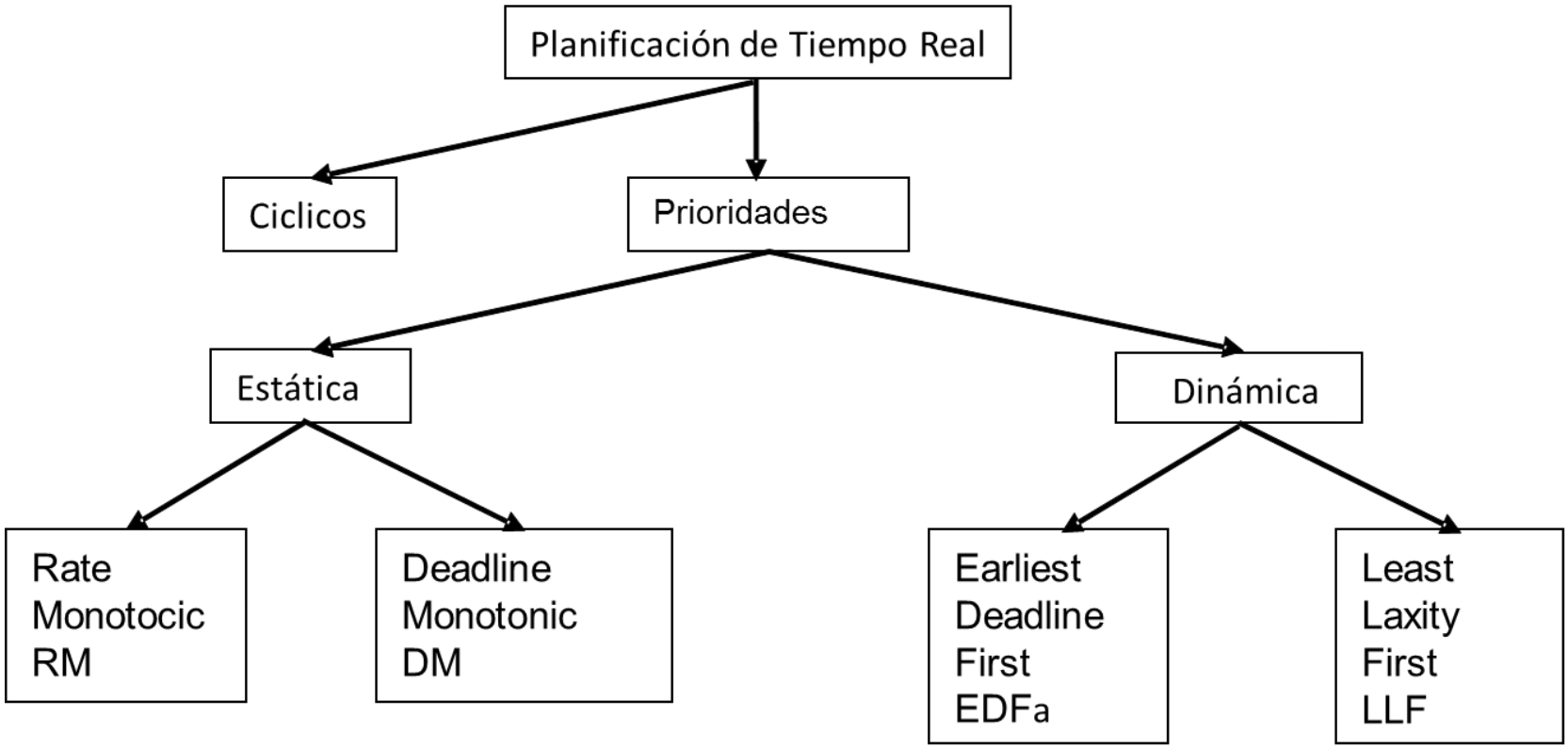
– Dinámicos sin tiempo real: minimizar tiempo de respuesta y maximizar uso de recursos.

– Sistemas de tiempo real: cumplir los límites temporales.

  - 5 tareas pueden realizarse de 5!=120 ordenes diferentes
  - Pero debo elegir una que cumpla restricciones temporales!

- En particular, el recurso que se planifica es el uso de procesador/es disponibles
- Un planificador debe producir planificaciones válidas, de forma que satisfagan las siguientes condiciones:
  - Cada procesador debe estar asignado como máximo a un solo trabajo en cada instante.
  - Cada trabajo está asignado como máximo a un solo procesador en cada instante
  - La cantidad total de tiempo asignada a c/trabajo debe ser igual a su t. de ejecución máximo o a su actual t.de ejecución
  - Se satisfacen todas las restricciones de precedencia y uso de recursos para acceder a los datos
- Con estas condiciones se da por supuesto que un trabajo no se ejecuta en paralelo en más de un procesador, o se consideran como trabajos distintos los que se ejecuten en procesadores distintos, aunque colaboren para realizar una función en conjunto

Planificación



Planificación

- Una planificación válida es posible si cada trabajo finaliza antes de su límite temporal
- Un conjunto de tareas es planificable de acuerdo con un algoritmo de planificación si cuando utiliza este algoritmo el planificador siempre produce planificaciones posibles
- Rendimiento de un algoritmo de planificación: habilidad de encontrar planificaciones posibles
- Algoritmo de planificación óptimo: siempre produce planificaciones posibles cuando existen en el conjunto de trabajos dados
- Si un planificador óptimo falla al encontrar una planificación posible, el conjunto de tareas no puede ser planificada con ningún otro algoritmo de planificación
- Puede ocurrir que un planificador no consiga encontrar una planificación posible para un conjunto de tareas porque no exista
- Puede ser aceptable que para algunos procesos (opcionales) no se cumplan sus restricciones temporales (trabajos que finalizan fuera de plazo, o no lleguen a ejecutarse)
- En este caso, un planificador elige entre minimizar la tardanza de los trabajos que se ejecutan fuera de plazo, o minimizar el número de tareas que no se ejecutan para que las que lo hacen si cumplan los plazos
- Estos dos criterios son opuestos: completar más trabajos aumentará la tardanza total y disminuir la tardanza dejará sin ejecutar más trabajos
- Una medida del rendimiento que tiene en cuenta ambos criterios es la razón de invalidez, que contempla el total de los trabajos inválidos, los que no se ejecutan y los que no terminan en plazo
- Por lo general, un planificador que admita trabajos opcionales debe minimizar la razón de invalidez

Planificación

- tareas : sucesión de trabajos que se repiten. Cuando la tarea comienza a ejecutarse comenzará a ejecutarse su primer trabajo y, cuando el último trabajo finalice, terminará la ejecución de la tarea.

- Un método de planificación tiene dos aspectos:

– Algoritmo de planificación, que determina el orden de acceso de las tareas a los recursos.

– Un método de análisis que permite calcular el comportamiento temporal del sistema:

- Se pueden comprobar si requisitos temporales están garantizados en **todos los casos posibles.**
- En general se estudia el **peor comportamiento posible.**

– Los algoritmos pueden ser estáticos y dinámicos.

Planificación

- Planificación Estática:

– Los requisitos temporales se analizan antes de la ejecución y se determina el orden en que estas se ejecutarán.

– Minimiza la sobrecarga en tiempo de ejecución.

– Menos flexible.

- Planificación dinámica:

– El orden de las tareas se decide durante la ejecución.

– El análisis también se realiza durante la ejecución.

- Prioridades Fijas y Procesos Totalmente Interrumpibles.

– La prioridad es un parámetro relacionado con la urgencia o la importancia de la tarea.

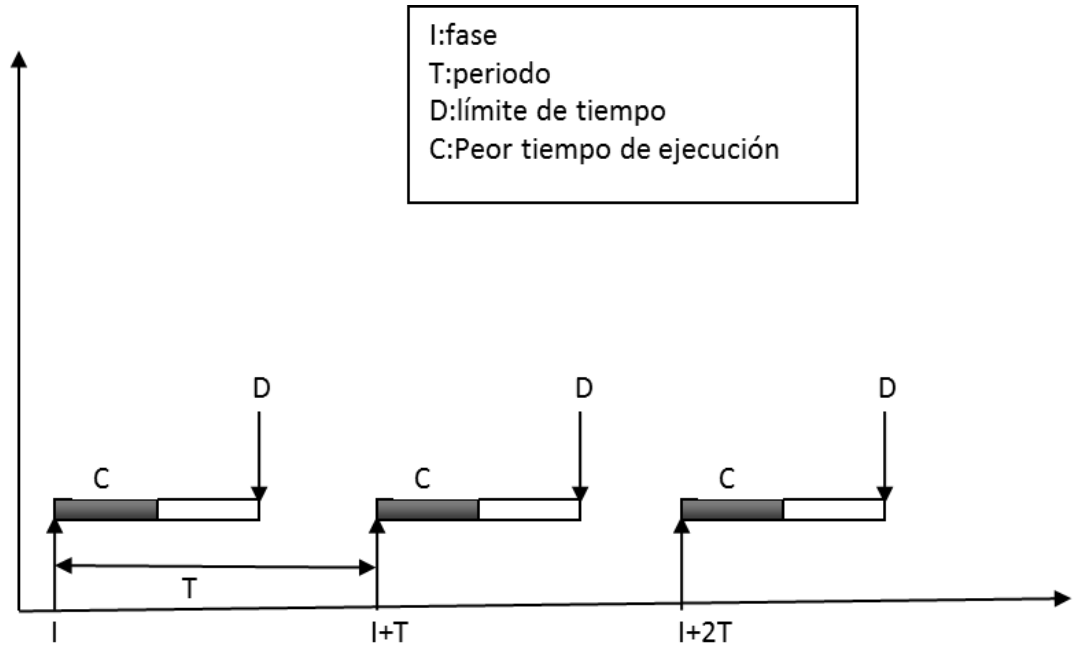
– Siempre se ejecuta la tarea lista con mayor prioridad.

- Es el algoritmo de mejor comportamiento en SOTR crítico, sobre todo en sobrecarga.

Planificación

- Modelo de tareas, de mas simple a mas complejo:

– El conjunto de tareas es estático.
– Todas las tareas son periódicas
– Las tareas son independientes
– Los límites de tiempo de todas las tareas son iguales a su periodo.
– El tiempo de ejecución máximo es conocido.



Ejecutivos cíclicos

- Si todas las tareas son periódicas, se puede confeccionar un plan de ejecución fijo en una tabla.
- Los ejecutivos cíclicos son la aproximación más tradicional a la construcción de sistemas de tiempo real.
- No se utiliza la concurrencia. Las distintas tareas se simulan en un programa secuencial, sin soporte del lenguaje o sistema operativo.
- Se trata de un esquema que se repite cada:

  $Tm=mcm(Ti)$ (ciclo principal)
- El ciclo principal se divide en ciclos secundarios, con período.

  $Ts \mid Tm=k \cdot Ts$
- En cada ciclo secundario se ejecutan las actividades correspondientes a determinadas tareas
- En el planificador solo consulta tabla y el procesador ejecute las tareas siempre en el mismo orden.
- Se coloca “a mano” el orden de ejecución de los trabajos durante un hiperperiodo fuera del tiempo de ejecución.

Ejecutivos cíclicos

Tarea	T	C
A	4	0.5
B	4	1.5
C	8	1
D	16	2

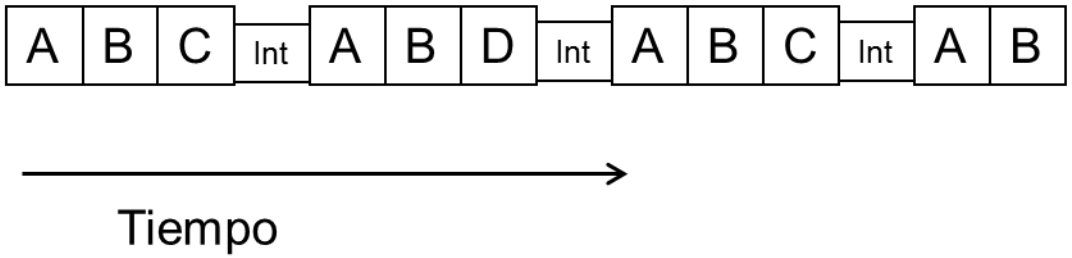
- Tabla de llamadas a procedimientos (T periodo y C tiempo ejecución peor caso)
- Cada procedimiento representa parte del código de un proceso
- El ciclo principal dura 16ms
- Hay 4 ciclos secundarios de 4ms.
- Cada 4ms. hay una interrupción y el planificador pone en ejecución la tarea correspondiente hasta recorrer la tabla
- Será planificable si la suma de los tiempos da menor a 16.

Ejecutivos cíclicos

El seudo código será:

```
Loop
        espera_interrupción;
        procedimiento_para_a;
        procedimiento_para_b;
        procedimiento_para_c;
espera_interrupción;
        procedimiento_para_a;
        procedimiento_para_b;
        procedimiento_para_d;
espera_interrupción;
        procedimiento_para_a;
        procedimiento_para_b;
        procedimiento_para_c;
espera_interrupción;
        procedimiento_para_a;
        procedimiento_para_b;
End loop;
```

Ejecutivos cíclicos



- Los procesos no existen en tiempo de ejecución: cada ciclo secundario es una secuencia de llamadas a procedimientos.
- Comparten un espacio de direcciones, por lo que pueden pasar datos entre ellos.
- Datos compartidos no necesitan ser protegidos (con un semáforo, por ejemplo), porque es imposible el acceso concurrente.
- Todos los periodos de los procesos deben ser múltiplos del tiempo del ciclo secundario

Ejecutivos cíclicos - Desventajas

- Esta última propiedad representa uno de los principales inconvenientes del enfoque del ejecutivo cíclico, junto a:
- La dificultad para incorporar procesos esporádicos.
- La dificultad de procesos con periodos grandes; el tiempo del ciclo principal es el máximo periodo acomodable sin planificación secundaria (un procedimiento en el ciclo principal que llame a un proceso secundario cada N ciclos secundarios).
- La dificultad a la hora de construir el ejecutivo cíclico.
- Cualquier proceso con un tiempo de cálculo notable podrá tener que ser dividido en un número fijo de procedimientos de tamaño fijo (lo que podría romper la estructura del código desde una perspectiva de la ingeniería de software, y por lo tanto ser propenso a error).
- Si puede construirse el ejecutivo cíclico, no será necesario ningún test de planificabilidad más.
- Sin embargo, en sistemas de alta utilización la construcción del ejecutivo es problemática. En sistemas periódicos simples el ejecutivo cíclico se mantiene como una estrategia de implementación apropiada.

Planificadores basados en prioridades

- En tiempo de ejecución se ejecuta la tarea de más prioridad activa

La asignación de prioridades puede ser:

- Estática: Prioridad de cada tarea constante durante toda la vida del sistema que se calcula antes de su ejecución. Se ejecutan por su prioridad. La prioridad de un proceso se deriva de sus requisitos de temporización, no de su importancia para el correcto funcionamiento o para la integridad del sistema.
- Dinámica: Las prioridades varían con el tiempo según unas determinadas reglas. El planificador puede ser expulsivo (apropiativo , preemtive) o no expulsivo. Será expulsivo si una tarea de más prioridad, al activarse, puede quitarle la CPU a otra de menor prioridad. Si el planificador es no expulsivo, cuando una tarea toma la CPU no la deja hasta que termine o la cede voluntariamente.

- En general, los esquemas apropiativos mejoran la reacción de los procesos de alta prioridad, y por lo tanto se les prefiere.
- En la planificación por prioridades, las tareas alternarán entre distintos estados según el control que tengan de la CPU.

Planificación con prioridades

- Sistema será planificable (planificación posible) si cada trabajo finaliza antes de su límite temporal
- Un conjunto de tareas es planificable de acuerdo con un algoritmo de planificación si cuando utiliza este algoritmo el planificador siempre produce planificaciones posibles
- La forma que tenemos de medir el rendimiento de un algoritmo de planificación en las aplicaciones de tiempo real es por su habilidad de encontrar planificaciones posibles en una determinada aplicación cuando estas existen.
- Un algoritmo de planificación es óptimo si utilizando este algoritmo el planificador siempre produce planificaciones posibles cuando existen en el conjunto de trabajos dados.
- Puede ocurrir que un planificador no consiga encontrar una planificación posible para un conjunto de tareas. Tal vez porque esta no exista.

Planificadores con prioridades estáticas:

- Rate Monotonic (RM).
- Deadline Monotonic (DM).

Planificadores con prioridades dinámicas:

- Earliest Deadline First (EDF)
- Least Laxity First (LLF).

Rate Monotonic

- Es un esquema óptimo de asignación de prioridades (tasa monotónica)
- A cada proceso se le asigna una única prioridad basada en su periodo: mayor prioridad para periodos menores
- Es óptima porque si un conjunto de procesos puede ser planificado con un esquema de prioridades estático (utilizando una planificación apropiativa basada en prioridades), entonces el conjunto dado también se puede planificar con un esquema de planificación de tasa monotónica.
- Prioridad al más frecuente => Plazo = periodo.
- La asignación de prioridades más altas a las tareas más frecuentes (período más corto) es óptima entre todas las asignaciones con prioridades fijas (Liu & Layland,1973).

Deadline Monotonic

- Extensión de Rate Monotonic (DM) donde las tareas pueden tener un plazo relativo menor que su período.

- Idéntica a la Rate Monotic, pero ahora los plazos de finalización pueden ser distintos a los periodos (menores o iguales):

$$D_i \leq T_i$$

- La asignación de prioridades se hace en orden inverso al plazo de finalización, de forma que la tarea con plazo de finalización más breve tenga mayor prioridad.
- Prioridad al más urgente => Plazo ≤ periodo.
- La asignación de prioridades más altas a las tareas más urgentes (plazo de respuesta más corto) es óptima
- DM es óptimo, esto es si un algoritmo de planificación con prioridad estático puede planificar un conjunto de tareas con plazos diferentes a sus periodos, entonces DM también planificara este conjunto de tareas.
- La factibilidad de un conjunto de tareas con plazo menor a su plazo puede ser garantizada usando el algoritmo de planificación Rate-Monotonic, reduciendo los periodos de las tareas a su plazo relativo.

POSIX 1003.1b

- Estándar para introducir características de tiempo real en UNIX
- Las tareas pueden ser periódicas o no.
- Las tareas se programan de acuerdo a la prioridad y a la política de las tareas.
- La prioridad es estática.
- La política puede ser SCHED_RR, SCHED_FIFO o SCHED_OTHERS
- Describe cómo el planificador elige una tarea cuando varias tareas tienen el mismo nivel de prioridad.
- Tareas SCHED_RR y SCHED_FIFO debe tener prioridades que oscilan entre 255 y 1
- Nivel de prioridad 0 se reserva para tareas SCHED_OTHERS.
- El nivel de prioridad más alto es 255.
- En esta política cada tarea tiene una prioridad asignada, y el planificador elige para ejecutar aquella tarea que tiene mayor prioridad, de entre los que están listos para ejecutar.
- Cuando hay tareas que tienen el mismo nivel de prioridad el planificador utiliza una política para elegir que tarea ejecutar.

Políticas implementadas

SCHED_FIFO: Este es el nombre de la política de planificación expulsora con prioridades que usa orden FIFO para determinar el orden en el que se ejecutan las tareas de la misma prioridad.

SCHED_RR (Round - Robin (turno rotatorio): Este es el nombre de la política de planificación expulsora con prioridades que usa un esquema cíclico para planificar tareas de la misma prioridad. Un proceso o hilo se ejecuta hasta que acaba o es bloqueado o hasta que su quantum de tiempo ha expirado; si un proceso/hilo es expropiado por un proceso de mayor prioridad, es colocado a la cabeza de la cola de ejecución para esa prioridad sin embargo, si su quantum expira, será colocado al final.

(SCHED_OTHER) : tiene un comportamiento definido por la implementación, que se definió para permitir la compatibilidad con implementaciones preexistentes

Test de planificabilidad (factor de utilización)

- Planificadores con prioridades estáticas,
- Si se cumple la siguiente condición, todos los N procesos cumplirán sus tiempos límite
- Un conjunto de n tareas será planificable bajo el Rate-Monotonic si se cumple:

$$\sum_{i=1}^{n} \frac{Ci}{Ti} \leq N(2^{1/N} - 1)$$

- La sumatoria calcula el factor de utilización
- El test es suficiente pero no necesario. Si un conjunto de procesos pasa el test, entonces cumplirá todos los tiempos límite; si lo falla, puede o no fallar en tiempo de ejecución.
- Sólo aporta una respuesta del tipo sí/no. No da ninguna indicación sobre los tiempos de respuesta de los procesos.

Planificadores con prioridades dinámicas. Earliest Deadline First (EDF)

- Asignar en cada instante la prioridad más alta al proceso cuyo plazo de respuesta está más próximo. Se asignan las prioridades dando preferencia a las tareas con plazo de finalización más próximo.
- La prioridad de una tarea se calcula en tiempo de ejecución.
- La prioridad de una tarea puede cambiar a lo largo de su ejecución y en diferentes activaciones.
- Prioridades dinámicas

Earliest Deadline First es un método de planificación óptimo bajo las siguientes restricciones:

- Tareas independientes
- Planificación expulsiva
- Sistemas monoprocesador

Least Laxity First (LLF)

- Se asignan las prioridades dando preferencia a las tareas con menor holgura, (plazo menos tiempo de procesamiento restante) de la tarea.

Bibliografía

- 1 Sistemas de Tiempo Real y Lenguajes de Programación, 3° edición, Alan Burns, Andy Wellings.
- 2 The Cheddar project: A free real time scheduling analyzer, http://beru.univ-brest.fr/~singhoff/cheddar/
- 3 Sistemas de Tiempo Real, Ingeniería en Informática, José Luis Villarroel Salcedo. Departamento de Informática e Ingeniería en Sistemas, Centro Politécnico Superior Universidad de Zaragoza. http://webdiis.unizar.es/~joseluis/STR.pdf.
- 4 Real-Time System SCHEDULING, N Audsley, A. Burns, Department of Computer Science, University of York, UK.
- 5 C. L. Liu and J. W. Layland, “Scheduling Algorithms for Multiprogramming in a Hard Real-Time Environment, Journal of ACM, Vol 20, No. 1, pp. 46-61,1973.
- 6 Planificación en Tiempo real, Universidad de Valencia http://www.uv.es/gomis/Apuntes_SITR/Planificacion.pdf
- 7 Rate Monotonic vs EDF, Giorgio C. Buttazzo University of Pravia Italy.

  Real-Time Systems, 29, 5–26, 2005 2005 Springer Science + Business Media, Inc. Manufactured in The Netherlands.
- 8 Tests de faisabilité basés sur le taux d’occupation CPU des principaux algorithmes d’ordonnancement temps réel. Martin Hervé, Bothorel Sebastien. Maitrise Informatique-année 2003-2004.